

Laboratorio SOLID

Presentado por:

Pablo Esteban Olaya Arias

Andres Felipe Rosada Mosquera

Profesora:

Liliana Marcela Olarte Mesa



Universidad Nacional de Colombia
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial
Ingeniería de Software II - Grupo 2
Bogotá D.C., Colombia
2026-I

Código Desarrollado

Enlace GitHub: <https://github.com/N1ghtlyC0de/SOLID-Workshop>

Estructura de la Implementación

```
Python
solid-lab/
├── interfaces/
│   ├── profession.py    # Abstracción Profession (DIP)
│   └── actions.py       # IHealable, IBuildable, IPaintable, etc. (ISP)
├── models/
│   └── human.py         # Solo name y age (SRP)
├── professions/
│   ├── doctor.py
│   ├── engineer.py
│   ├── artist.py
│   ├── teacher.py
│   └── musician.py     # Reto de extensión (OCP)
├── main.py
└── README.md
```

Discusiones

Sección 3

1. ¿Por qué es mejor separar la lógica de profesiones de la entidad *humana*?

Porque evita que la clase `Human`` esté anidada a la lógica de las clases para las profesiones. Esto asegura el cumplimiento de la SRP, ya que limita a la clase de los humanos a manejar exclusivamente el nombre y edad de la persona, y dicha clase no se verá afectada en el caso que se quieran modificar los métodos pertenecientes a las profesiones.

2. ¿Cómo permite tu diseño la fácil extensión?

Debido a que se cumple con el principio de “abierto a extensión, cerrado a modificación”, añadir nuevas profesiones no requiere de la modificación de las clases ya existentes. Esto es de gran ayuda, ya que garantiza que el sistema pueda crecer de manera ordenada al mantener la estabilidad del código ya existente, lo que reduce el riesgo de introducir errores si se llega a añadir nuevas funcionalidades (por ejemplo, la adición de la profesión `Musician``).

3. ¿Qué violaría LSP en este diseño?

El LSP se vería comprometido si una subclase de `Profession`` altera el comportamiento esperado por el sistema. Por ejemplo, al retornar tipos de datos inesperados, lanzar excepciones no controladas o imponer nuevas condiciones que el método `perform_action()`` no anticipaba.

4. ¿Cómo mejora ISP la flexibilidad del código?

La flexibilidad del código se ve mejorada ya que la ISP implica que el sistema evita la carga de métodos innecesarios a clases que no las requieren. Esto da paso a que el sistema se comporte de manera modular, en el cual cada clase implementa únicamente las interfaces que necesita, y una nueva profesión puede combinar varias interfaces si lo requiere.

5. ¿Cómo ayuda DIP con pruebas y extensión de su simulación?

Ayuda al permitir que el código sea más flexible y desacoplado, lo que facilita cambiar implementaciones sin afectar otras partes de la base de código. Esto significa que se pueden inyectar implementaciones falsas (*mocks*) de la interfaz `Profession`` sin necesidad de modificar la clase `Human``.

Sección 4

Principio	Función
SRP	La lógica musical quedó aislada en su propia clase <code>`Musician`</code> . No se mezcló con <code>`Human`</code> .
OCP	Se agregó <code>`Musician`</code> sin editar ninguna de las profesiones existentes, ni alterar la clase <code>`Human`</code> .
LSP	<code>`Musician`</code> se usa exactamente igual que <code>`Doctor`</code> en la simulación. Puede pasarse al constructor de <code>`Human`</code> de forma transparente y el método <code>`perform_action()`</code> corre sin romper el programa.
ISP	Solo se añadió <code>`IMusical`</code> y <code>`Playable`</code> , la cual no es una interfaz monolítica. Evita que las demás profesiones tengan que arrastrar el método <code>`playMusic()`</code> .
DIP	<code>`Human`</code> ya dependía de la abstracción <code>`Profession`</code> ; solo se le pasó la instancia <code>`Musician()`</code> .

Sección 5

1. ¿Qué desafíos enfrentaron?

Dado que era nuestra primera experiencia aplicando de manera estricta los principios SOLID, el proceso de verificación y comprobación de que el diseño cumpliera con cada uno de ellos resultó parcialmente arduo. En particular, diseñar interfaces pequeñas, significativas y no excesivamente fragmentadas fue un reto no trivial. Asimismo, garantizar que nuevas profesiones pudieran añadirse sin necesidad de modificar el código existente exigió disciplina y consistencia en la aplicación de los principios de diseño.

2. ¿Cómo ayudaron los principios SOLID?

A medida que íbamos avanzando en el laboratorio, la implementación de los principios SOLID se fue volviendo más fácil de hacer, y poco a poco pudimos ir notando las ventajas de aplicarlos correctamente. Por ejemplo, el código se volvió más legible, modular y fácil de mantener. Además, la separación de responsabilidades y el desacople entre clases facilitaron la incorporación de nuevas profesiones sin afectar las existentes, hechos que lo hicieron más ameno.

3. ¿Cómo podrían mejorar aún más su diseño?

Se puede extender la funcionalidad de la simulación de forma que las profesiones sean más rigurosas y detalladas, y tengan muchos más comportamientos. También se podría permitir que hayan humanos con más de una profesión, ya que enriquecería el nivel de realismo del sistema. Estas mejoras se pueden implementar gracias a la implementación de los principios SOLID, ya que el diseño actual es modular y permite extensiones.